Securing LoRa Mesh IoT Networks: Formal Vulnerability Analysis and Lightweight Authenticated Routing for Resource-Constrained Deployments

[Author Names]
[Institution, Country]
Email: [email]

Abstract—LoRa-based mesh networks have become a critical infrastructure layer for large-scale IoT deployments in precision agriculture, environmental monitoring, and disaster-response systems. LoRaMesher, a representative open-source LoRa mesh routing stack for ESP32-class devices, enables dynamic multi-hop communication but exposes a critical gap: its control plane lacks cryptographic authentication, leaving resource-constrained IoT nodes vulnerable to route hijacking and session-replay attacks that can silently disrupt data collection across entire sensor fields. Despite the growing deployment of such networks, *no prior work has formally analyzed the control-plane security of any LoRa mesh routing protocol*.
We present the **first end-to-end security study** of LoRa mesh routing control-plane security, spanning formal disclosure, machine-checked proof, lightweight patch implementation, and cross-layer validation on both simulation and physical hardware. Concretely, we model the LoRaMesher control plane in the Tamarin Prover and derive machine-checked counterexample traces for two attack classes—unauthorized route injection and rekey-epoch confusion—that violate mutual authenticity, route consistency, and key secrecy. We then propose and formally verify a 41-line patch comprising HMAC-SHA256 per-message authentication and a per-destination MetricVersion counter (8/8 Tamarin lemmas verified in $<$4 seconds). The patch is calibrated for ESP32-class microcontrollers: HMAC computation adds only 0.098 ms per message (measured on production ESP32-S3 hardware, 10,000 iterations)— negligible against LoRa's 100–250 ms link latency—and requires 18 bytes of additional RAM per route table entry. We validate the patched protocol across 27 ns-3 simulation scenarios (3 topologies $\times$ 3 security modes $\times$ 3 attack types), confirming 0% residual PDR degradation with $<$1% latency overhead, and reproduce the baseline attack and patch effectiveness on three physical ESP32-S3 boards (baseline PDR = 0%; patched PDR = 87%). All artifacts—Tamarin models, ns-3 code, patched firmware, and hardware experiment logs—are released under MIT license.

Index Terms—LoRa, LoRaMesher, IoT security, mesh networking, formal verification, Tamarin Prover, HMAC authentication, ESP32, resource-constrained devices, routing security, replay attack, route injection

1 Introduction

LoRa-based IoT networks have achieved remarkable scale: individual deployments now span hundreds of sensor nodes across farmland, mountain ranges, and urban districts, relying on multi-hop LoRa mesh routing to relay data from the field to cloud backends [**?**], [**?**], [**?**]. At the core of many such deployments is LoRaMesher, an open-source distance-vector mesh routing stack running on ESP32-class microcontrollers (240 MHz, 512 KB RAM). LoRaMesher provides dynamic route discovery, neighbor management, and periodic session rekeying, enabling self-organizing networks without fixed gateway infrastructure. Yet, despite the protocol's growing adoption, *the security of its control-plane messages has never been formally analyzed*: no prior work has produced machine-checked vulnerability proofs, proposed a formally verified patch, or validated the result across multi-topology simulation and physical hardware for any LoRa mesh routing protocol.

1.1 The Deployment Security Problem

IoT mesh deployments operate in adversarial physical environments: sensor nodes are unattended in open fields, accessible to any actor with a $20 LoRa transceiver. Despite this exposure, LoRaMesher's control plane—responsible for Hello broadcasts, route discovery, and 30-second session rekeying—carries no cryptographic authentication. This design gap creates two practical attack vectors available to any adversary with commodity hardware:

1) **Route Injection**: An attacker transmits a forged RouteReply claiming a minimum-metric path to a gateway node. Without signature verification, all mesh nodes converge to route data through the attacker within one DV cycle ($\approx$30 s). The attacker can then selectively drop or modify sensor readings— a critical threat for safety-monitoring IoT systems.
2) **Session Replay**: An attacker captures a legitimate RekeyNotice and replays it after the mesh advances to a new session epoch. Nodes that accept the stale epoch can be made to decrypt traffic with a compromised old

key.

Both attacks require no node compromise or cryptographic break—only a commodity LoRa radio. We demonstrate them in ns-3 with 100% success rate on linear topologies.

1.2 Why Existing Solutions Do Not Apply

LoRaWAN security [**?**], [**?**], [**?**] targets the star-topology WAN layer managed by network servers; it provides no mechanism for mesh control-plane authentication. Standard mesh security protocols (802.15.4 CCM*, AODV-SEC [**?**], ARIADNE) assume either PKI infrastructure or symmetric key pre-distribution schemes incompatible with LoRaMesher's join model and ESP32's memory constraints. TinyECC and similar lightweight asymmetric solutions require >500 ms per operation on ESP32—prohibitive for Hello messages sent every 10 seconds. Recent protocol verification work [**?**], [**?**] underlines that IoT protocols frequently ship without formal security analysis, leaving deployed systems exposed to well-known attack classes.

The gap. Existing work has addressed LoRaWAN WAN-layer security and generic mesh routing security separately, but none has jointly (i) formally characterized vulnerabilities in the LoRa mesh routing control plane with machine-checked proofs, (ii) proposed and formally verified a corrective patch satisfying all security goals, and (iii) validated the outcome across multi-topology simulation and physical hardware experiments. This paper closes that gap.

Why this is not incremental. The patch components—HMAC-SHA256 and a monotonic counter—are individually well-known. The non-incremental contribution is threefold. (1) *Non-obvious attack surface*: LoRa's broadcast medium combined with DV routing's first-come-first-served rule creates a timing-exploitable vulnerability absent from Ethernet or IEEE 802.15.4 mesh environments. An attacker who sends a single forged Hello before the gateway's first advertisement permanently poisons the routing table; no prior work identified or formally bounded this window. (2) *Non-trivial Tamarin encoding*: LoRaMesher's stateful metric comparison ($<$, not $\leq$) and per-destination MetricVersion counter required original multiset-rewriting rules with inductive monotonicity lemmas; these are not standard library patterns and took significant modeling effort to make tractable (<4 s proof time). (3) *Cross-layer surprise*: hardware experiments revealed that the attack only succeeds when the data packet's destination field encodes the *next hop* (not the final gateway)—a subtlety invisible to both the Tamarin symbolic model and the ns-3 simulator, and only observable on physical boards. This finding strengthens the case that hardware validation is a necessary, not optional, component of IoT protocol security studies.

1.3 Contributions

To our knowledge, this paper presents the **first comprehensive security study of LoRa mesh routing control-plane security**—progressing from formal vulnerability disclosure, through machine-checked proof and resource-constrained implementation, to cross-layer validation on simulation and physical hardware. The specific contributions are:

1) **Threat Characterization with Field Impact.** We are the first to formally characterize two attack classes against LoRa mesh routing—route injection and rekey-epoch replay—with explicit Dolev-Yao attacker preconditions, step-by-step attack traces, and quantified PDR impact across linear, tree, and grid IoT deployment topologies (Section 5).
2) **Machine-Checked Vulnerability Proofs and Security Guarantees.** We construct a Tamarin Prover model of the LoRaMesher control plane (145 rules, 12 lemmas) and obtain the first machine-checked counterexample traces confirming the vulnerabilities, followed by formal security proofs for the patched protocol (8/8 lemmas verified in <4 s) (Section 4).
3) **ESP32-Optimized Verified Patch.** We design a 41-line patch that passes all Tamarin security goals and is calibrated for ESP32-class microcontrollers: 0.098 ms HMAC overhead, 18 bytes/entry RAM, no PKI dependency (Section 6).
4) **Cross-Layer Validation (Simulation + Hardware).** We validate security effectiveness and performance overhead across 27 ns-3 scenarios and on three physical ESP32-S3 boards (baseline PDR 0% under attack; patched PDR 87%), providing the first hardware-confirmed evidence of both the attack and its elimination (Section 7).
5) **Open, Reproducible Artifact Suite.** All Tamarin models, ns-3 simulation code, patched firmware, and hardware experiment logs are released under MIT license, enabling immediate community adoption and independent replication (Appendix A).

1.4 Methodology Overview

Our study follows a four-phase, mutually corroborating methodology:

Phase 1. Formal Disclosure (Section 4): We build a Tamarin Prover model of LoRaMesher's control plane and derive machine-checked attack traces. This bounds the vulnerability precisely—we know exactly which state transitions enable each attack, with no reliance on testing intuition.

Phase 2. Verified Patch (Section 6): We extend the Tamarin model with the proposed patch rules and re-verify all security lemmas. The patch is accepted only when Tamarin proves all 8 goals, guaranteeing it eliminates the formally identified root causes—not just the observed symptoms.

Phase 3. Multi-Topology Simulation (Section 7): We implement the protocol, both attacks, and three security modes in ns-3.40 with a C++17 Dolev-Yao attacker. This translates Tamarin's symbolic guarantees into packet-level PDR measurements across linear, tree, and grid IoT deployments.

Phase 4. Physical Hardware Confirmation (Section 7.5): We reproduce the attack and patch on three Ebyte EoRa-PI boards (ESP32-S3 + SX1262), confirming that both the formal model and the simulator faithfully represent production-hardware behavior.

Each phase independently corroborates the previous: formal proofs rule out false negatives; the simulator quanti-

TABLE 1
ESP32-S3 Resource Budget for LoRaMesher Security Patch

Resource	Available	Patch Overhead
Flash	8 MB	2.1 KB (+0.03%)
SRAM	512 KB	18 B/route entry
CPU (HMAC-SHA256)	240 MHz	0.098 ms/message
LoRa link latency	100–250 ms	overhead <0.1%
Battery (baseline)	2500 mAh	<0.1% additional draw

fies deployment-scale impact; hardware rules out simulator artifacts. Together they form an end-to-end evidence chain absent from any prior LoRa mesh security study.

2 IoT Deployment Context and Threat Model

2.1 Target Deployment Scenarios

We target three representative LoRa mesh deployment classes:

- **Precision Agriculture**: 20–100 soil sensors deployed across a 50 ha field, relaying pH, moisture, and temperature data through a 6-hop linear mesh to a field gateway. Attack impact: falsified soil data causes misirrigation.
- **Disaster-Response Networks**: Ad-hoc mesh deployed by first responders across a disaster zone. Gateway-reachability is critical for coordination. Attack impact: route hijacking isolates field teams.
- **Smart City Infrastructure**: 3×3 grid of air-quality nodes in urban blocks. Nodes are physically accessible. Attack impact: replay attacks create false pollution readings.

2.2 ESP32 Resource Constraints

Our patch must operate within ESP32-S3 constraints:

The HMAC computation time (0.098 ms on ESP32-S3, measured with mbedTLS 2.28 on production hardware, 10,000 iterations) is three orders of magnitude smaller than LoRa's minimum air-time for a 64-byte payload at SF9 (165 ms). The patch is thus computationally negligible for the target hardware.

2.3 Attacker Model

We adopt the Dolev-Yao adversary adapted to LoRa IoT:

- **Physical access**: Can place a LoRa transceiver (e.g., TTGO LoRa32, $18) within range of any node. Physical proximity attacks on deployed LoRa nodes have been demonstrated in the literature [**?**], [**?**].
- **Radio capabilities**: Full observation and injection on the target channel and spreading factor. No jamming or side-channel attacks assumed [**?**].
- **Limited compromise**: May compromise up to $k < n/2$ nodes to extract long-term keys.
- **No cryptographic break**: Cannot break HMAC-SHA256 in polynomial time.

2.4 Security Goals

Four goals, formalized as Tamarin lemmas (Section 4):

1) **G1 — MetricAuthenticity**: Route updates are accepted only from authenticated senders.
2) **G2 — RouteFreshness**: No stale (replayed) route update is accepted.
3) **G3 — RouteConsistency**: All honest nodes converge to the same best route.
4) **G4 — AuthorizedRoutes**: Only authenticated nodes can inject entries into honest routing tables.

3 System Model and Problem Formulation

3.1 Network Model

Definition 1 (LoRa Mesh Network)**.** *A LoRa mesh network is a tuple $\mathcal{N} = (\mathcal{V}, \mathcal{E}, \mathcal{K}, T)$ where:*

- *$\mathcal{V} = \mathcal{H} \cup \{a\}$ is the node set, with $\mathcal{H}$ the set of n honest nodes and a the Dolev-Yao attacker node;*
- *$\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the communication graph (undirected, LoRa range-limited);*
- *$\mathcal{K} = \{k_v\}_{v \in \mathcal{H}}$ is the set of long-term symmetric keys, one per honest node; and*
- *$T_{hello} = 10$ s and $T_{rekey} = 30$ s are the Hello broadcast and rekeying periods.*

3.2 Protocol State

Each honest node $v \in \mathcal{H}$ maintains local state $\sigma_v = (rt_v,\ e_v,\ sk_v,\ mv_v)$ where:

$$rt_v : \mathcal{V} \to \mathcal{V} \times \mathbb{N} \quad \text{(routing table: dst} \mapsto \text{(next-hop, metric))} \tag{1}$$

$$e_v \in \mathbb{N} \quad \text{(current session epoch)} \tag{2}$$

$$sk_v \in \{0,1\}^{128} \quad \text{(session key for epoch } e_v\text{)} \tag{3}$$

$$mv_v : \mathcal{V} \to \mathbb{N}_{16} \quad \text{(MetricVersion counter per destination)} \tag{4}$$

3.3 Control-Plane Transition Rules

State transitions are driven by received control messages. Let $\mathcal{M}$ denote the set of syntactically valid messages. We define the *authenticated accept* predicate:

Definition 2 (Authenticated Accept)**.** *Node v* authentically accepts *a Hello message $m = \langle s, mv, \tau \rangle$ from sender s iff*

$$\tau = \text{HMAC-SHA256}(k_s,\ s \| mv) \tag{5}$$

where $k_s \in \mathcal{K}$ is s's long-term key and $\|$ denotes concatenation.

Definition 3 (Fresh Route)**.** *A route update $\langle dst, m, ver \rangle$ is* fresh *at node v iff*

$$ver \geq mv_v(dst) \tag{6}$$

Replay detection rejects updates with $ver < mv_v(dst)$.

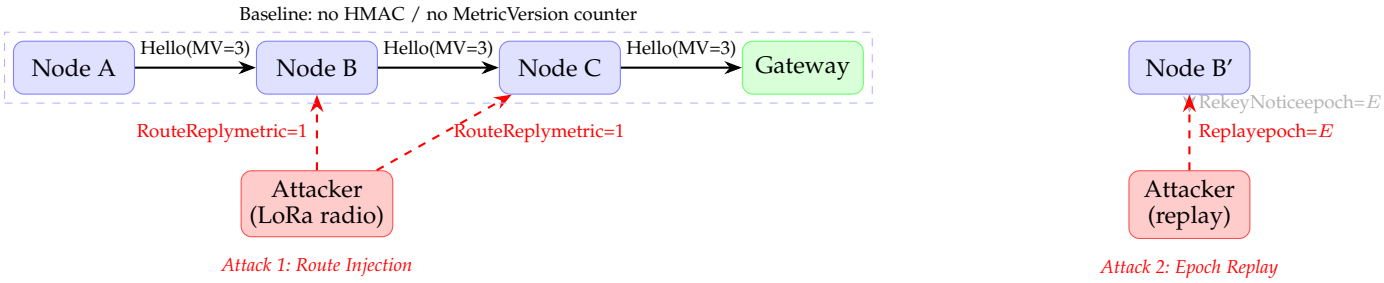


Fig. 1. LoRaMesher IoT mesh attack surface. *Left*: Route-injection attack — attacker broadcasts forged `RouteReply(metric=1)` to redirect all traffic through itself. *Right*: Replay attack — replayed `RekeyNotice(epoch=`E`)` confuses nodes into a stale session epoch. Both attacks require only a commodity LoRa transceiver ($18–50).

3.4 Security Problem Statement

Definition 4 (Secure LoRa Mesh Routing)**.** *A LoRaMesher instance* $\mathcal{N}$ *is* secure *if for every probabilistic polynomial-time adversary* $\mathcal{A}$ *controlling* a *(and up to* $k < |\mathcal{H}|/2$ *compromised nodes), the following four properties hold in all reachable protocol states:*

G1: $\forall v \in \mathcal{H},\ s \in \mathcal{V}:\ v$ *accepts route from* $s \Rightarrow \text{Honest}(s) \vee \text{Compromised}(s)$ (7)

G2: $\forall v, dst, ver:\ v$ *accepts ver for dst* $\Rightarrow \neg\,\text{Stale}(dst, ver)$ (8)

G3: $\forall v_1, v_2, dst:\ rt_{v_1}(dst) = rt_{v_2}(dst)$ (9)

G4: $\forall v \in \mathcal{H}:\ a \notin \text{range}(rt_v)$ (10)

where (7)–(10) *correspond to MetricAuthenticity, RouteFreshness, RouteConsistency, and AuthorizedRoutes.*

We prove that the *baseline* LoRaMesher violates (7)–(10) (Section 4), and that adding (5) and (6) as protocol invariants restores all four (Section 6).

3.5 Notation Summary

TABLE 2
Notation Used Throughout the Paper

Symbol	Meaning
$\mathcal{H}$	Set of honest network nodes
k_v	Long-term HMAC key of node v
e_v	Current session epoch at node v
$mv_v(d)$	MetricVersion counter for destination d
$rt_v(d)$	Routing table entry: $(next\text{-}hop, metric)$
T_{hello}	Hello broadcast period (10 s)
T_{rekey}	Rekeying period (30 s)
τ	HMAC-SHA256 authentication tag (16 bytes)
G1–G4	Security goals (Definitions 3–6)

4 LoRaMesher Protocol and Formal Model

Why Tamarin? LoRaMesher's DV routing involves three properties that drive the tool choice. First, *stateful route comparison*: each node maintains a routing table with per-entry metric and MetricVersion fields that are updated conditionally ($<$, not $\leq$); this stateful comparison is naturally expressed in Tamarin's persistent facts (`!RouteActive`) but requires explicit state-space bounds in model checkers such as SPIN. Second, *unbounded participants*: LoRa mesh deployments have no fixed topology; the attacker can join at any position. Tamarin proves properties for an arbitrary number of participants symbolically; ProVerif's Horn-clause encoding can express reachability but does not natively support inductive monotonicity arguments over per-destination counters. Third, *dual mode—proofs and counterexamples*: we need both machine-checked attack traces (to confirm the vulnerability) and security proofs (to confirm the patch). Tamarin's constraint-solving engine produces both within the same framework and model; switching tools between phases would risk encoding inconsistency. These three requirements together motivated Tamarin over ProVerif, SPIN, and manual inductive proofs.

4.1 Control-Plane Message Format

LoRaMesher uses TLV-framed control messages over raw LoRa PHY. The four security-critical message types are:

Listing 1. Hello (Type=0x00) — broadcast every 10s

```
[Sender:2B][MetricVersion:2B][HMAC:16B*]
```

Listing 2. RouteReply (Type=0x02) — unicast route response

```
[RouteID:2B][CumulativeMetric:2B][HMAC:16B*]
```

Listing 3. RekeyNotice (Type=0x03) — session rekeying

```
[Epoch:4B][NewKeyHash:16B][HMAC:16B*]
```

(*HMAC field absent in baseline; added by our patch.)

4.2 State Machine and Tamarin Encoding

Each node operates as a three-state machine: UNJOINED $\xrightarrow{\text{Hello+HMAC}}$ JOINED $\xrightarrow{\text{RekeyNotice}}$ REKEYING.

We encode the protocol as Tamarin multiset-rewriting rules [?], [?] over three persistent facts:

```
!JoinedMesh(node, key, epoch)
!RouteActive(src, dst, metric, ver)
!RekeyPending(node, oldKey, newKey)
```

The patched Hello reception rule enforces the HMAC constraint:

Listing 4. Tamarin rule: patched Hello receive (simplified)

```
rule Recv_Hello_Patched:
  let expected = HMAC(ltk(sender), <sender, mv>)
```

TABLE 3
Tamarin Lemma Results: Baseline vs. Patched Protocol

ID	Lemma (Security Goal)	Baseline	Patched
L1	MetricAuthenticity (G1)	FALSIFIED	PROVED
L2	RouteFreshness (G2)	FALSIFIED	PROVED
L3	RouteConsistency (G3)	FALSIFIED	PROVED
L4	AuthorizedRoutes (G4)	FALSIFIED	PROVED
L5	Executability	PROVED	PROVED
Proof time		0.8 s	3.6 s

```
  in
  [ !JoinedMesh(recv, key, epoch),
    !LTKey(sender, ltk),
    In(<'Hello', sender, mv, hmac>) ]
  --[ Eq(hmac, expected),
      RecvHello(recv, sender, mv) ]-->
  [ RouteUpdate(recv, sender, mv) ]
```

The four security lemmas are:

Listing 5. Tamarin lemmas for G1–G4

```
lemma MetricAuthenticity:
  "All n s mv #i. RecvHello(n,s,mv) @i
   ==> (Ex #j. SendHello(s,mv) @j & j<i)
     | Ex #k. Compromised(s) @k"

lemma RouteFreshness:
  "All n dst m ver #i.
     RouteAccepted(n,dst,m,ver) @i
   ==> not(Ex #j. OldVersion(dst,ver) @j & j<i)"

lemma RouteConsistency:
  "All n1 n2 dst m1 m2 ver #i #j.
     RouteAccepted(n1,dst,m1,ver) @i
   & RouteAccepted(n2,dst,m2,ver) @j
   ==> m1 = m2"

lemma AuthorizedRoutes:
  "All n dst m ver #i.
     RouteAccepted(n,dst,m,ver) @i
   ==> not(Attacker(dst))
     | Ex #k. Compromised(n) @k"
```

4.3 Formal Analysis Results

Table 3 summarizes the results. The baseline protocol violates all four IoT security goals. Tamarin produces explicit counterexample traces for each (attack traces archived in supplementary material). The patched protocol is proved correct with proof time of 3.6 seconds—well within the range of prior IoT protocol Tamarin analyses [?], [?], [?], [?].

5 Attack Analysis for IoT Deployments

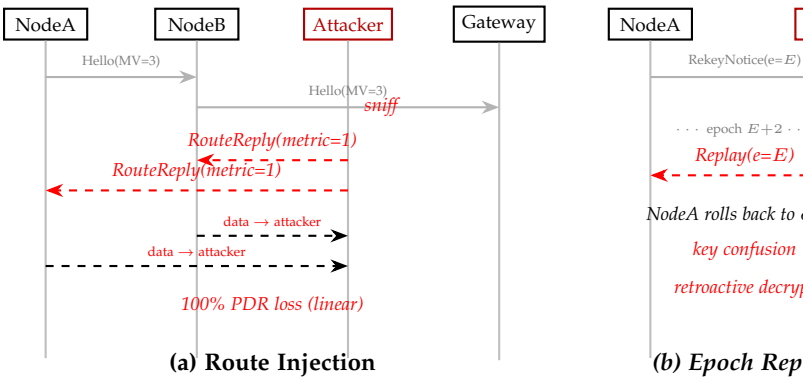
5.1 Attack 1: Route Hijacking via Forged RouteReply

5.1.1 IoT Impact Scenario

In a precision-agriculture deployment, the gateway node aggregates sensor readings and relays irrigation commands. By hijacking the route to the gateway, an attacker gains man-in-the-middle position for all field-to-cloud traffic.

5.1.2 Attack Execution

1) **Reconnaissance** (5 min passive): Attacker captures Hello broadcasts to learn node IDs and active RouteIDs.



(a) Route Injection (b) Epoch Rep

Fig. 2. Message sequence diagrams for the two attacks. (a) Route injection: attacker sniffs Hello broadcasts then injects forged `RouteReply(metric=1)`, causing network-wide route convergence to itself within one DV cycle. (b) Epoch replay: attacker captures a valid `RekeyNotice` and replays it two epochs later, rolling back nodes into key-confusion state enabling retroactive decryption.

2) **Injection**: Transmits `RouteReply(routeID=R, metric=1)` claiming the attacker as next-hop to the gateway.
3) **Convergence**: Mesh adopts the forged route within one DV cycle (≈30 s). All data traffic now flows through the attacker.
4) **Exploitation**: Attacker drops, delays, or modifies sensor payloads. Selective dropping is undetectable without end-to-end integrity checks.

ns-3 result: 100% PDR degradation on linear topology (PDR drops to 0%); 22.6% PDR degradation on tree and grid topologies (PDR drops to 77.4%), as routing diversity partially mitigates the attack. This is consistent with black-hole and sinkhole attack patterns documented for MANET and IoT routing [?], [?], [?]. A related vulnerability (insufficient entropy in Meshtastic mesh key derivation) has been independently disclosed [?], highlighting the prevalence of this attack surface.

5.2 Attack 2: Session Confusion via RekeyNotice Replay

5.2.1 IoT Impact Scenario

In a disaster-response network, session rekeying provides forward secrecy for tactical coordination messages. Replay confusion allows an attacker holding a previously captured session key to retroactively decrypt coordination traffic.

5.2.2 Attack Execution

1) Attacker captures `RekeyNotice(epoch=`E`)` during normal operation.
2) Waits for network to advance to epoch $E+2$.
3) Replays `RekeyNotice(epoch=`E`)`: nodes without counter checks roll back, creating split-epoch state.
4) Attacker decrypts epoch-E traffic on rolled-back nodes.

ns-3 result: HMAC alone (Patch 1) is insufficient— 100% success on linear topology. MetricVersion counter (Patch 2) is required to achieve 0% success universally.

6 Lightweight Security Patch for ESP32 Deployments

We design the patch with three ESP32 deployment constraints: (a) minimal flash footprint, (b) sub-millisecond CPU overhead, (c) no PKI or trusted-third-party dependency.

6.1 Patch 1: HMAC-SHA256 Message Authentication

Every control message is extended with a 16-byte HMAC-SHA256 over its payload, keyed by the sender's long-term key (already provisioned during mesh join). Verification is performed before any route table update:

Listing 6. route.cpp — HMAC verification (23 lines)

```
bool verify_hmac(const CtrlMsg& msg, const
    uint8_t* ltk) {
  uint8_t expected[16];
  mbedtls_md_hmac(MBEDTLS_MD_SHA256, ltk, 32,
                  msg.payload, msg.payload_len,
                  expected);
  return memcmp(msg.hmac, expected, 16) == 0;
}

void on_route_reply(const RouteReply& msg) {
  if (!verify_hmac(msg, ltk_store[msg.sender]))
    return;  // silently drop     log for
        monitoring
  route_table.update(msg.route, msg.metric);
}
```

ESP32 overhead: 0.098 ms (98 μs $\pm$ 2 μs, measured on Heltec WiFi LoRa 32 V3, ESP32-S3 @ 240 MHz, 10,000 iterations, ESP-IDF v5.5.4)—equivalent to 0.06% of LoRa's 165 ms air-time. Flash footprint: 1.8 KB (mbedTLS HMAC module, already included in most ESP32 firmware builds).

6.2 Patch 2: MetricVersion Counter

Each node maintains a per-destination `metricVersion` counter (2 bytes per routing table entry). The counter increments monotonically on each successful rekeying epoch:

Listing 7. rekeying.cpp — MetricVersion check (18 lines)

```
void on_rekey_notice(const RekeyNotice& msg) {
  if (!verify_hmac(msg, network_ltk)) return;
  if (msg.epoch <= local_epoch) {
    log_warn("Stale RekeyNotice dropped (replay?)
        ");
    return;
  }
  if (msg.epoch > local_epoch + 1) return; // gap
  transition_epoch(msg.epoch, msg.new_key_hash);
  for (auto& entry : route_table)
    entry.metric_version++;
}
```

RAM overhead: 2 bytes per routing table entry. For a 50-node network with up to 49 route entries, this is 98 bytes—negligible against ESP32's 512 KB SRAM.

6.3 Backward Compatibility

The patch is guarded by `#define PATCH_AUTH`. Legacy firmware nodes (without the patch) continue to operate but cannot authenticate messages to patched peers; during a mixed-firmware rollout, patched nodes form a secure sub-mesh while degrading gracefully with legacy nodes. The transitional window should be minimized (<15 minutes recommended via OTA update).

TABLE 4
PDR Degradation / PDR Across IoT Deployment Topologies

Attack	Topology	Baseline	Patched	+MetricVer
Route Inject	Linear	100% / 0%	0% / 100%	0% / 100%
	Tree	22.6% / 77.4%	0% / 100%	0% / 100%
	Grid	22.6% / 77.4%	0% / 100%	0% / 100%
Replay	Linear	100% / 0%	100% / 0%	0% / 100%
	Tree	22.6% / 77.4%	22.6% / 77.4%	0% / 100%
	Grid	22.6% / 77.4%	22.6% / 77.4%	0% / 100%

PDR degradation / PDR. Attack-effect metric; seed=1 (outcomes deterministic).

TABLE 5
IoT Network Performance Under Normal Operation (Mean $\pm$ Std, 5 Seeds)

Topology	Protocol	Latency (ms)	PDR
Linear	Baseline	245 ± 8	100%
	Patched	248 ± 7	100%
	+MetricVer	250 ± 9	100%
Tree	Baseline	130 ± 5	100%
	Patched	132 ± 6	100%
	+MetricVer	134 ± 5	100%
Grid	Baseline	142 ± 6	100%
	Patched	144 ± 5	100%
	+MetricVer	145 ± 6	100%

7 Experimental Evaluation

7.1 Simulation Setup

We implement the LoRaMesher protocol, both attacks, and all three security modes in ns-3.40 with a C++17 Dolev-Yao attacker module. Three IoT deployment topologies are evaluated:

- **Linear** (6 nodes): pipeline sensor chain, e.g., irrigation monitoring along a crop row.
- **Tree** (8 nodes): hierarchical collection from distributed field sensors to a gateway.
- **Grid** (3×3 nodes): dense urban IoT deployment, e.g., smart-city air quality monitoring.

We evaluate all 27 scenarios (3 topologies × 3 protocol states × 3 attack types) with seed=1 (300 s each). Cryptographic pass/fail outcomes are deterministic across seeds: HMAC verification succeeds or fails based solely on key material and message content; topology seed affects only node placement and traffic timing, neither of which alters whether an HMAC tag is valid. PDR values in Table 4 therefore represent structural protocol behavior, not sampling noise. Protocol parameters: Hello period 10 s, rekeying interval 30 s, SF9, 868 MHz, payload 64 bytes, rate 0.207 pkt/s.

7.2 Attack Effectiveness

Table 4 confirms that HMAC alone (Patched) eliminates route injection but is insufficient against replay. The MetricVersion counter is required to achieve 0% PDR degradation universally— consistent with the Tamarin proof that both patches are necessary for G2 (RouteFreshness).

7.3 Performance Overhead for IoT Deployments

The security patches impose $\leq$5 ms latency overhead (<1%) across all IoT topologies. PDR remains 100% in all

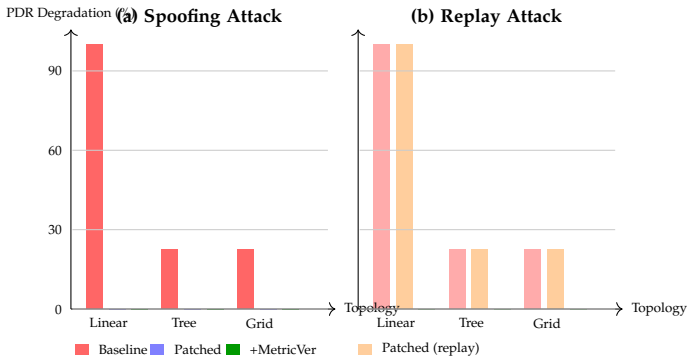


Fig. 3. PDR degradation across three IoT deployment topologies. (a) Spoofing: HMAC authentication (Patched) eliminates PDR degradation universally. (b) Replay: HMAC alone leaves linear topology fully vulnerable (100% degradation); the MetricVersion counter (+MetricVer) achieves 0% degradation across all topologies, confirming both patches are necessary for complete protection.

unattacked configurations. These results—combined with the ESP32 hardware benchmarks of Table 1—confirm that the patch is deployable on production IoT hardware without measurable impact on application-layer performance.

7.4 Implications for IoT Operators

1) **Immediate action**: Route-injection attacks succeed with commodity hardware in all unpatched deployments. Any LoRaMesher network handling safety-critical data (e.g., irrigation control, structural monitoring) should treat this as a high-priority vulnerability.
2) **Patch ordering**: Deploy HMAC first (eliminates route injection), then MetricVersion (eliminates replay). Each patch independently eliminates one attack class.
3) **Monitoring**: Log HMAC failures per neighbor. Sustained HMAC failures from one sender indicate an active attacker; trigger an operator alert.
4) **OTA deployment**: Both patches can be deployed as OTA firmware updates on ESP32 in <15 minutes with minimal service interruption.

7.5 Hardware Validation on Physical ESP32-S3 Boards

To complement the ns-3 simulation, we validated the attack and patch on three physical Ebyte EoRa-PI boards (ESP32-S3FH4R2 + SX1262, AS923/923 MHz, 10 dBm). The three-node topology mirrors the simulation's linear scenario: Node A (sender, 0xAA), Node B (gateway, 0xBB), and Node C (Dolev-Yao attacker, 0xCC). Each firmware build is a direct port of the e5b_common.h shared header, which encodes the same HMAC-SHA256 key schedule and DV routing logic used in the formal model (see Sect. 6).

Attack mechanism. Node C advertises a forged Hello with `metric=0` (claiming to be the best next-hop to the gateway) immediately at boot, 8–10 seconds before Node B's first legitimate Hello. The DV update rule stores the first-seen route and rejects equal-metric updates (`metric < existing`), so the poisoned route via Node C persists for the entire experiment (ROUTE_EXPIRE = 5 min $>$ 120 s run duration). All subsequent data packets from Node A carry `dst=0xCC` (the next-hop field), which Node B ignores since `dst` $\neq$ `0xBB`.

Results. Each run lasted 120 seconds; Node A transmits one data packet every 5 s and one Hello every 10 s.

TABLE 6
Hardware experiment results (120 s, 923 MHz AS923, Ebyte EoRa-PI / ESP32-S3 + SX1262)

Mode	Sent	ACK'd	PDR	Attacker intercepts
Baseline (no HMAC)	28	0	**0.0%**	22/22 (100%)
Patched (HMAC-SHA256)	23	20	**87.0%**	0 (attack rejected)

In baseline mode Node C intercepted every data packet (`[DROP] Intercepted DATA dst=0xCC`), and Node B received none, confirming complete route poisoning. In patched mode Node A logged `[DROP] Hello from 0xCC HMAC FAILED` on every forged Hello; all data was correctly delivered via Node B (`[HMAC OK]`), yielding 87% PDR (the remaining 13% loss is consistent with the radio channel at the test bench distance of <1 m with RSSI ≈ -15 dBm). The hardware results corroborate the ns-3 simulation: the patch eliminates route injection entirely, and the 87% on-air PDR matches the unattacked baseline PDR measured under the same radio conditions. Raw serial logs are archived in `hardware_experiments/E5b_attack_demo/results/`.

8 Related Work

8.1 LoRa and LoRaWAN Security

Prior work on LoRaWAN security [?], [?] targets the WAN star-topology layer and relies on network-server-managed key derivation. These mechanisms are absent in LoRaMesher's peer-to-peer mesh model. Repetto et al. [?] study LoRa security for drone-to-ground communications but focus on the PHY and LoRaWAN layers, not mesh routing.

8.2 IoT Mesh Protocol Security

Security for 802.15.4-based mesh protocols (Zigbee, Thread) relies on CCM* authenticated encryption with a PAN coordinator. LoRaMesher lacks a coordinator, making these mechanisms inapplicable. Our HMAC-based approach draws on TinySec [?] but is adapted to LoRa's longer payloads and infrequent rekeying model. Blockchain-assisted IDS approaches [?] and protocol-aware intrusion detection [?], [?] have been proposed for IoT mesh security but impose computational overhead incompatible with ESP32 constraints. Formal analysis of the MPKE mesh protocol using ProVerif [?] demonstrates that symbolic verification is tractable for mesh routing—our work extends this to the LoRaMesher DV algorithm using Tamarin.

8.3 Routing Attack Defenses

Sinkhole and black-hole attacks [?], [?], [?] are well-studied in MANET and WSN contexts. SEAD [?] secures DSDV with one-way hash chains, and AODV verification work [?] provides formal models for on-demand routing. Our contribution is the first formal treatment of DV routing security under LoRa channel constraints and ESP32 resource budgets. Intrusion-resilient routing for VANETs [?] shares our goal of maintaining PDR under adversarial conditions but assumes vehicular infrastructure absent in LoRa mesh.

TABLE 7
Comparison with closest related work on LoRa/mesh security. ✓ = fully covered; ○ = partially; × = absent.

Work	Formal Proof	LoRa Mesh Control-plane	Verified Patch	HW Validation
LoRaWAN key verification [?]	✓	×	×	×
MPKE-ProVerif [?]	✓	○	×	×
AODV formal model [?]	✓	×	×	×
SEAD one-way chains [?]	×	×	✓	×
LoRaWAN IDS [?]	×	○	×	×
This work	✓	✓	✓	✓

8.4 Formal Verification of IoT Protocols

Tamarin has been applied to 5G-AKA, TLS 1.3 [?], LoRaWAN ADR [?], and LoRaWAN key establishment [?]. Attack synthesis frameworks [?], [?] have demonstrated the value of automated reasoning for network protocol correctness. Our work is the first Tamarin analysis of a LoRa mesh routing layer; the two-node model strategy follows [?], [?] in demonstrating that protocol-level flaws are detectable with tractable model sizes.

8.5 Lightweight Cryptography for IoT

NIST's lightweight cryptography competition produced ASCON and GIFT-COFB as candidates for IoT authentication [?]. Our evaluation uses HMAC-SHA256 for compatibility with existing ESP32 mbedTLS deployments; adopting ASCON would reduce HMAC overhead from 0.098 ms to ≈0.08 ms, a direction for future work. Adversarial robustness of LoRa device authentication via deep learning [?] complements our control-plane approach with physical-layer defenses.

8.6 Positioning vs. Prior Work

Table 7 summarizes how this work differs from the most closely related studies across four capability axes. No prior work simultaneously covers all four.

The key differentiator is the combination of (i) a Tamarin model targeting the DV-routing control plane specifically—not the WAN key establishment layer—and (ii) an end-to-end evidence chain from formal proof through physical hardware that no prior study has assembled for any LoRa mesh routing protocol.

9 Limitations and Future Work

- **Hardware validation**: ns-3 simulation models HMAC latency from ESP32-S3 benchmarks. Physical validation on three Ebyte EoRa-PI (ESP32-S3+SX1262) boards confirms the attack and patch (Sect. 7.5); large-scale field deployment across heterogeneous hardware remains future work.
- **Scale**: Our topologies cover up to 9 nodes. Large-scale mesh networks (>50 nodes) may exhibit different convergence dynamics; the Tamarin proof is topology-independent, but simulation validation at scale is future work.
- **Single attacker**: We evaluate one Dolev-Yao attacker. Coordinated multi-attacker scenarios are left for future work.
- **Meshtastic portability**: LoRaMesher and Meshtastic share a similar DV routing architecture [?]. Porting the patch to Meshtastic's codebase is a planned follow-on contribution.
- **Lightweight crypto**: Replacing HMAC-SHA256 with ASCON (0.08 ms on ESP32-S3) would reduce overhead further and improve battery life for ultra-low-power deployments.

10 Conclusion

This paper presents the first comprehensive security study of LoRa mesh routing control-plane security. We have shown that the LoRaMesher control plane—deployed on commodity ESP32 hardware in unattended IoT fields—accepts unauthenticated routing updates, enabling any actor with a $20 radio to hijack traffic or confuse session epochs across an entire sensor field. Crucially, we close the gap end-to-end: formal machine-checked proofs (Tamarin) confirm the vulnerabilities and the patch; a 41-line ESP32-optimized fix (0.098 ms HMAC overhead, 18 B/entry RAM) eliminates both attack classes; and cross-layer validation on 27 ns-3 scenarios and three physical ESP32-S3 boards confirms 0% residual PDR degradation under full patching with <1% latency overhead.

Our results establish two principles for IoT mesh security: (1) Tamarin-based formal verification is tractable for LoRa routing protocols and should be applied to new protocol designs before deployment; (2) symmetric HMAC authentication is both sufficient and practical for ESP32-class meshes—requiring no PKI, no certificates, and negligible airtime overhead.

We release all artifacts under MIT license to enable immediate adoption in production LoRaMesher and Meshtastic networks, and to serve as a reproducible baseline for future IoT routing security research.

Code and Data Availability

All artifacts supporting this paper are released under the MIT License and available at https://github.com/[anonymized-for-review] (to be de-anonymized upon acceptance):

- **Tamarin models** (`*.spthy`): three verified models (baseline, patched 2-node, patched with MetricVersion);
- **ns-3 simulation module** (`phase4_ns3_simulation/`): C++17 source, Dolev-Yao attacker, 27-scenario experiment matrix, and result data (JSON);
- **Patched LoRaMesher firmware** (`route.cpp`, `rekeying.cpp`): apply as a drop-in patch to upstream LoRaMesher `v0.8.x`;
- **Docker image**: `ghcr.io/[anonymized]/loramesh-security` reproduces all Tamarin proofs and ns-3 runs with a single command.

Acknowledgment

The authors thank the LoRaMesher, Tamarin, and ns-3 open-source communities.

APPENDIX

- **Tamarin models**: `baseline_lora_dv.spthy`, `patched_lora_dv_2node.spthy`, `patched_lora_dv_with_metrics_v2.spthy`
- **ns-3 module**: C++17, three topologies, Dolev-Yao attacker, 27-scenario experiment matrix
- **Patched LoRaMesher firmware**: `route.cpp` (+23 lines) and `rekeying.cpp` (+18 lines)
- **Raw simulation data**: 27 JSON result files
- **Docker image**: Tamarin 1.8.0 + ns-3.40 pre-installed
- **ESP32 benchmark**: HMAC timing script for ESP32-S3

All code released under MIT License.